Midterm Project Report

Name: Le Cao Son
Student ID: 22070749
Email: 22070749@vnu.edu.vn

October 19, 2026

Abstract

This report presents an experimental study of three classical computer vision tasks implemented with traditional methods only: image filtering, stereo-based 3D reconstruction, and multi-view image stitching. In Part A, Mean, Gaussian, Median, and Laplacian-sharpening filters are evaluated under Gaussian ($\sigma_n = 25$) and salt-and-pepper (5%) noise using PSNR, SSIM, MSE, and runtime. A 5×5 Gaussian filter ($\sigma = 1$) gives the best accuracy–cost trade-off for Gaussian noise (28.25 dB), whereas a 3×3 Median filter is clearly superior for impulse noise (32.12 dB). In Part B, Semi-Global Block Matching (SGBM) yields denser disparity maps than Block Matching (BM), with 81.2% versus 69.9% valid pixels, at about 2.4× the runtime. In Part C, SIFT, ORB, and AKAZE are compared inside a RANSAC-based homography pipeline: SIFT attains the highest inlier ratio (85.6%), while AKAZE achieves a comparable ratio in less than half of the time.

1 Introduction

Despite the dominance of deep learning, traditional computer vision methods remain valuable because they are grounded in explicit mathematical models, require no training data, and expose interpretable parameters whose effects can be analysed directly. This report documents a study of three core tasks solved exclusively with such methods: spatial-domain image filtering and enhancement (Part A), 3D reconstruction from a rectified stereo pair (Part B), and panorama stitching from four overlapping photographs (Part C). All experiments are implemented in Python with OpenCV, NumPy, SciPy, and scikit-image.

Each task is motivated by a practical need. Image filtering is a preprocessing step that suppresses noise and enhances structure before higher-level analysis. Stereo reconstruction recovers depth from the geometric relationship between two viewpoints, which is fundamental to robotics and navigation. Image stitching extends the limited field of view of a camera by aligning overlapping images with projective transformations. The report is organised as follows: each of Sections 2–4 describes the methodology, presents the implementation and results, and closes with a quantitative and qualitative comparative analysis; Section 5 summarises the findings and limitations.

2 Part A: Image Filtering

2.1 Methodology

Experimental setting. A grayscale photograph of Albert Einstein [5] (longest side resized to at most 512 pixels) serves as the clean reference I. Two degradations are simulated with a fixed random seed (42) so that the results are reproducible. *Additive white Gaussian noise* is generated as

$$\tilde{I}(x,y) = \mathrm{clip}_{[0,255]}\big(I(x,y) + n(x,y)\big), \qquad n \sim \mathcal{N}(0,\sigma_n^2),\ \sigma_n = 25, \tag{1}$$

and *salt-and-pepper noise* with density $p = 0.05$ is generated as

$$\tilde{I}(x,y) = \begin{cases} 0, & \text{with probability } p/2, \\ 255, & \text{with probability } p/2, \\ I(x,y), & \text{with probability } 1-p. \end{cases} \tag{2}$$

Four classical filters are applied with kernel sizes $k \in \{3,5,7\}$, where $a = (k-1)/2$.

(a) Mean (box) filter. A linear filter that replaces each pixel with the average of its $k \times k$ neighbourhood:

$$I'(x,y) = \frac{1}{k^2} \sum_{i=-a}^{a} \sum_{j=-a}^{a} \tilde{I}(x+i,\, y+j). \tag{3}$$

All neighbours receive equal weight, so edges are blurred together with the noise.

(b) Gaussian filter. A linear filter whose weights decay with distance from the centre. The kernel is the sampled Gaussian, normalised to unit sum over the window:

$$G(x,y) = \frac{1}{Z} \exp\!\left(-\frac{x^2+y^2}{2\sigma^2}\right), \qquad Z = \sum_{i=-a}^{a} \sum_{j=-a}^{a} \exp\!\left(-\frac{i^2+j^2}{2\sigma^2}\right), \tag{4}$$

and the output is the convolution $I' = G * \tilde{I}$. The kernel is separable, $G(x,y) = g(x)\,g(y)$, so the cost per pixel is $O(k)$ instead of $O(k^2)$. Two scales, $\sigma \in \{1.0, 2.0\}$, are examined to separate the influence of σ from that of the window size.

(c) Median filter. A non-linear rank filter:

$$I'(x,y) = \mathrm{median}\{\tilde{I}(x+i,\, y+j) \ :\ -a \le i,j \le a\}. \tag{5}$$

An impulse of extreme value occupies at most one rank in the sorted window and is therefore discarded when it is fewer than half of the window samples, while a step edge is preserved because the median selects an existing grey level instead of an average.

(d) Laplacian sharpening. The Laplacian is the isotropic second-order derivative operator

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}, \tag{6}$$

which responds strongly at intensity discontinuities. With `cv2.Laplacian`, a window of size $k > 1$ is built from Sobel second derivatives; for $k = 3, 5, 7$ the sums of absolute coefficients are $\|L_k\|_1 = 16,\ 112,\ 1200$ (versus 8 for the classical 4-neighbour kernel $\left[\begin{smallmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{smallmatrix}\right]$).

To make the sharpening strength comparable across kernel sizes, every kernel is rescaled to $\tilde{L}_k = (8/\|L_k\|_1)\, L_k$; for $k = 3$ this gives the effective kernel $\left[\begin{smallmatrix} 1 & 0 & 1 \\ 0 & -4 & 0 \\ 1 & 0 & 1 \end{smallmatrix}\right]$. Because the central coefficient is negative, the sharpened image is

$$I_{\text{sharp}} = \text{clip}_{[0,255]}\left(I - c\,(\tilde{L}_k * I)\right), \qquad c = 0.5, \tag{7}$$

computed in `float32` so that negative values and values above 255 are handled before clipping and conversion back to `uint8`. The Laplacian also amplifies high-frequency noise, so its behaviour is examined both on the noisy image directly and after a denoising step.

Evaluation metrics. Every output is compared with the clean image I using

$$\text{MSE} = \frac{1}{MN}\sum_{i=1}^{M}\sum_{j=1}^{N}\left(I(i,j) - \hat{I}(i,j)\right)^2, \tag{8}$$

$$\text{PSNR} = 10\log_{10}\frac{255^2}{\text{MSE}} \quad [\text{dB}], \tag{9}$$

$$\text{SSIM}(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \tag{10}$$

where $\hat{I}$ is the processed image, μ, σ^2, σ_{xy} are local means, variances, and covariance, $C_1 = (0.01 \cdot 255)^2$, and $C_2 = (0.03 \cdot 255)^2$. SSIM is computed with the default scikit-image settings (7×7 uniform window) and averaged over the image. The execution time of each configuration is the median of five repetitions measured with `time.perf_counter`.

2.2 Implementation and Results

Following [1], the filters are implemented with `cv2.blur` (Mean), `cv2.GaussianBlur` (Gaussian), `cv2.medianBlur` (Median), and `cv2.Laplacian` with the rescaling described above (sharpening). Figures 1 and 2 show, for each noise type, the original image, the noisy image, the output of each denoiser (fixed reference setting $k = 5$, $\sigma = 1.0$), and the result of applying Laplacian sharpening ($k = 3$, $c = 0.5$) to that denoised output. The red square in the first column marks the high-contrast region that is enlarged $4\times$ in Figures 3 and 4, which compare all kernel sizes and both values of σ.

From visual inspection, the Mean filter removes the grain but visibly softens edges, and the 7×7 window blurs fine structure considerably. The Gaussian filter gives a similar degree of denoising with a more natural appearance, and its effect grows with both k and σ. Under Gaussian noise the Median filter produces a smoothed, slightly "painted" look. Under salt-and-pepper noise, the 3×3 Mean and Gaussian filters leave faint grey blotches where each impulse has been spread over its neighbourhood (Figure 4), whereas the 3×3 Median filter removes the impulses cleanly while keeping the edge sharp; at 7×7 the Median filter begins to round small structures. Applied directly to a noisy image, Laplacian sharpening makes the noise much more prominent: in the zoomed salt-and-pepper row every impulse is surrounded by a bright/dark ring. The same operator on the *clean* image (Figure 5) increases local contrast but produces overshoot (halo) along strong edges, which becomes more visible as c increases or when the kernel is larger.



Figure 1: Results under Gaussian noise ($\sigma_n = 25$). Columns: original, noisy, denoised (Mean, Gaussian, Median, 5×5), and denoised followed by Laplacian sharpening (3×3, $c = 0.5$).



Figure 2: Results under salt-and-pepper noise ($p = 0.05$), with the same layout as Figure 1.

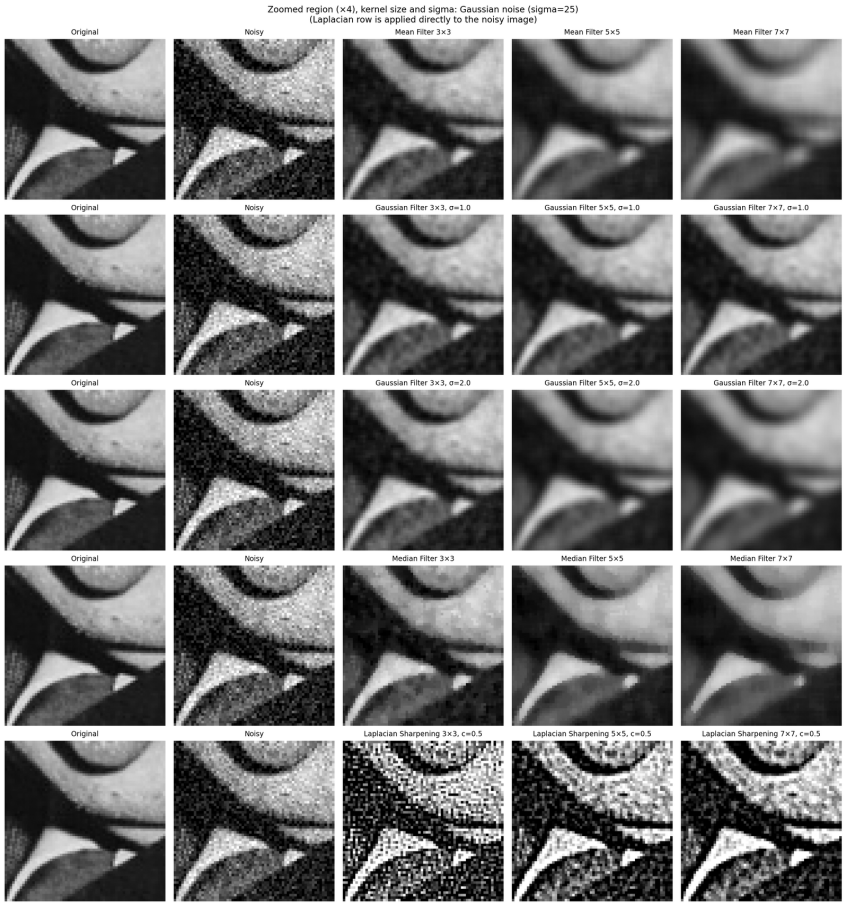


Figure 3: Zoomed edge region (×4) under Gaussian noise for kernel sizes 3×3, 5×5, 7×7. The last row applies Laplacian sharpening directly to the noisy image.

2.3 Comparative Analysis

Tables 1 and 2 report PSNR, SSIM, MSE, and execution time for all settings under the two noise types, and Figure 6 plots PSNR and SSIM against the kernel size. Since PSNR is a monotonic function of MSE, the two columns always rank the filters identically.

Gaussian noise. All four linear and non-linear denoisers improve PSNR by roughly 5.6–7.8 dB over the noisy baseline (20.49 dB). The Gaussian filter is the strongest: the 7×7, $\sigma = 1.0$ setting reaches 28.26 dB and SSIM 0.6994, but the 5×5, $\sigma = 1.0$ setting is only 0.01 dB lower (28.25 dB, SSIM 0.6964) at half the runtime (0.292 versus 0.589 ms), making it the best trade-off. The Mean filter degrades monotonically as the window grows ($27.67 \rightarrow 27.04 \rightarrow 26.06$ dB), which indicates that over-smoothing of edges outweighs the additional noise reduction beyond 3×3. For the Gaussian filter, a larger σ lowers PSNR when $k \geq 5$ ($28.25 \rightarrow 27.59$ dB at $k = 5$) because the effective support becomes wider, whereas at $k = 3$ the choice of σ is almost irrelevant (27.81 versus 27.76 dB) since the small window truncates the kernel towards a box shape. The Median filter performs worse than the linear filters here (26.23–26.80 dB, with the lowest SSIM at 3×3), consistent with the fact that for Gaussian samples the sample median is a less efficient estimator than the sample mean (its asymptotic variance is $\pi/2$ times larger).

Salt-and-pepper noise. The ranking is reversed. The 3×3 Median filter raises PSNR from 17.84 to 32.12 dB with SSIM 0.8963, outperforming the best linear setting (Gaussian

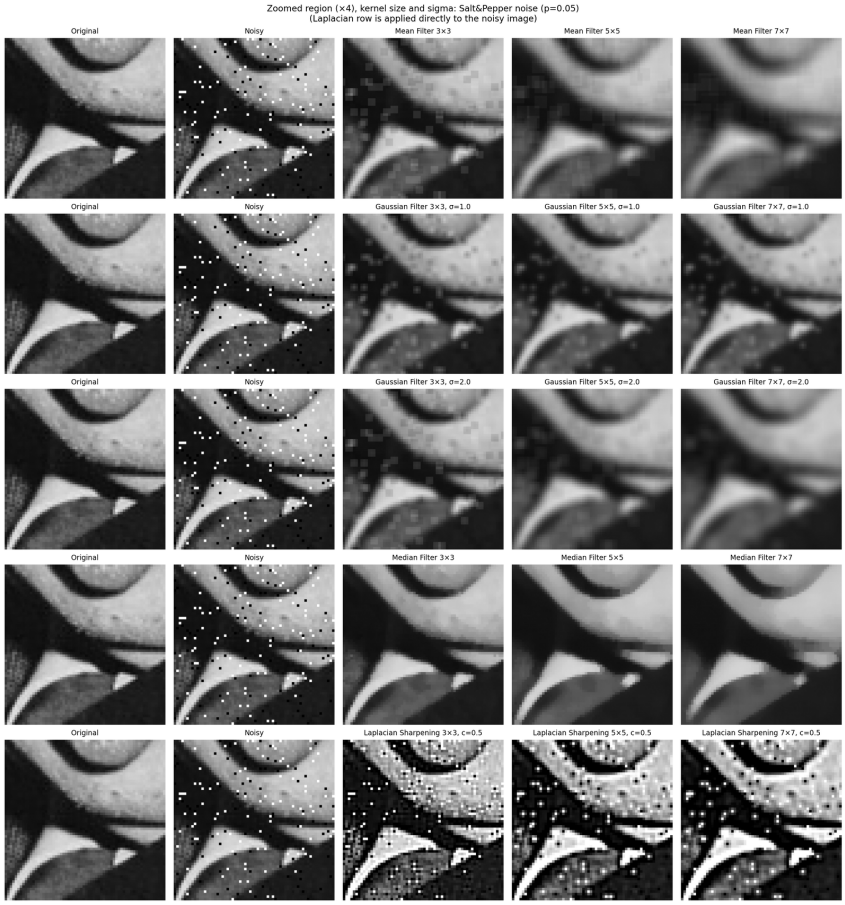


Figure 4: Zoomed edge region (×4) under salt-and-pepper noise, with the same layout as Figure 3.

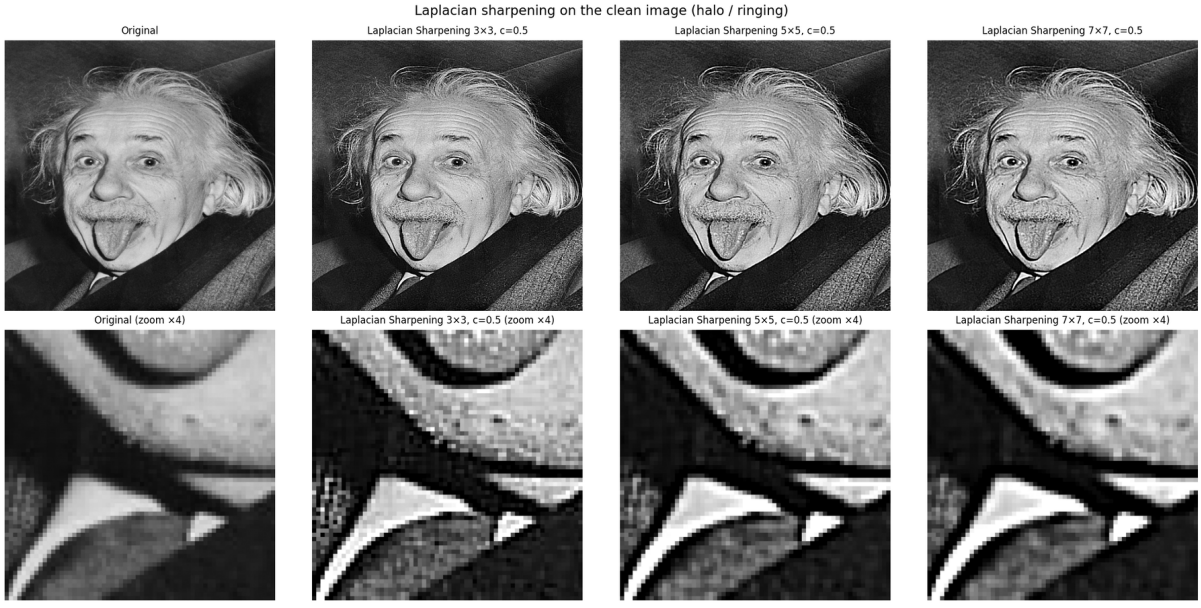


Figure 5: Laplacian sharpening ($c = 0.5$) applied to the clean image for $k = 3, 5, 7$ (top: full image, bottom: 4× zoom). Bright and dark rims along edges are the halo (ringing) artefact.

Filter	**Setting**	**PSNR (dB)**	**SSIM**	**MSE**	**Time (ms)**
Noisy (no filter)	–	20.49	0.3244	581.23	–
Mean	3×3	27.67	0.6522	111.26	**0.146**
Mean	5×5	27.04	0.6694	128.60	0.165
Mean	7×7	26.06	0.6441	160.94	0.422
Gaussian	3×3, $\sigma = 1.0$	27.81	0.6565	107.59	0.177
Gaussian	3×3, $\sigma = 2.0$	27.76	0.6572	108.91	0.179
Gaussian	5×5, $\sigma = 1.0$	28.25	0.6964	97.34	0.292
Gaussian	5×5, $\sigma = 2.0$	27.59	0.6978	113.33	0.256
Gaussian	7×7, $\sigma = 1.0$	**28.26**	**0.6994**	**97.02**	0.589
Gaussian	7×7, $\sigma = 2.0$	27.12	0.6984	126.11	0.712
Median	3×3	26.38	0.5693	149.79	0.475
Median	5×5	26.80	0.6230	135.87	3.206
Median	7×7	26.23	0.6251	155.08	11.385
Laplacian sharp.	3×3, $c = 0.5$	11.99	0.1188	4109.85	0.997
Laplacian sharp.	5×5, $c = 0.5$	14.88	0.1990	2114.99	2.387
Laplacian sharp.	7×7, $c = 0.5$	16.05	0.2349	1613.57	2.066

Table 1: Quantitative results under Gaussian noise ($\sigma_n = 25$). Bold marks the best denoising value in each column; Laplacian rows are applied directly to the noisy image.

Filter	**Setting**	**PSNR (dB)**	**SSIM**	**MSE**	**Time (ms)**
Noisy (no filter)	–	17.84	0.3737	1068.73	–
Mean	3×3	25.63	0.5780	178.06	**0.132**
Mean	5×5	26.04	0.6103	161.71	0.153
Mean	7×7	25.45	0.6079	185.48	0.425
Gaussian	3×3, $\sigma = 1.0$	25.55	0.5856	181.31	0.169
Gaussian	3×3, $\sigma = 2.0$	25.68	0.5831	176.02	0.167
Gaussian	5×5, $\sigma = 1.0$	26.33	0.6228	151.24	0.248
Gaussian	5×5, $\sigma = 2.0$	26.42	0.6360	148.23	0.268
Gaussian	7×7, $\sigma = 1.0$	26.39	0.6260	149.28	0.542
Gaussian	7×7, $\sigma = 2.0$	26.26	0.6530	153.77	0.576
Median	3×3	**32.12**	**0.8963**	**39.90**	0.497
Median	5×5	28.63	0.7766	89.10	3.249
Median	7×7	27.03	0.7046	128.83	11.209
Laplacian sharp.	3×3, $c = 0.5$	15.70	0.2935	1749.84	1.953
Laplacian sharp.	5×5, $c = 0.5$	15.47	0.2887	1843.64	6.557
Laplacian sharp.	7×7, $c = 0.5$	15.73	0.2999	1737.52	7.044

Table 2: Quantitative results under salt-and-pepper noise ($p = 0.05$), with the same conventions as Table 1.

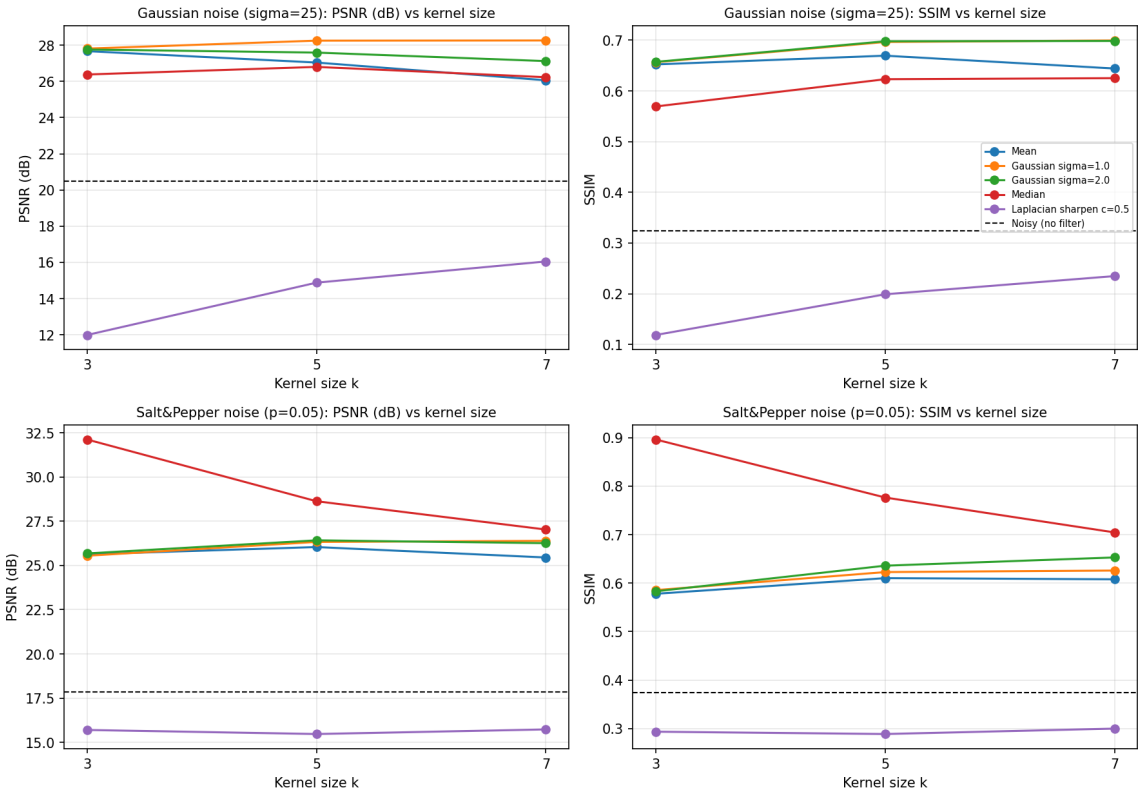


Figure 6: PSNR and SSIM as functions of the kernel size for each filter and noise type. The dashed line is the noisy image.

5×5, $\sigma = 2.0$: 26.42 dB, SSIM 0.6360) by about 5.7 dB. Linear filters cannot reject an outlier of value 0 or 255: they only dilute it, so larger windows are required (Mean peaks at 5×5), at the cost of blur. The Median filter, in contrast, becomes *worse* with a larger window (32.12 $\rightarrow$ 28.63 $\rightarrow$ 27.03 dB), since impulses are already removed at 3×3 and the additional window only erases genuine detail.

Computational cost. The Mean and Gaussian filters are very cheap and grow slowly with k (Gaussian $\sigma = 1$: 0.177 $\rightarrow$ 0.292 $\rightarrow$ 0.589 ms for $k = 3, 5, 7$). The Median filter grows much faster (0.475 $\rightarrow$ 3.206 $\rightarrow$ 11.385 ms), reaching about 27$\times$ the cost of the Mean filter at 7×7 (Gaussian-noise case), because it requires a rank selection over k^2 samples rather than a weighted sum.

Laplacian sharpening and the denoise–sharpen pipeline. Applied directly to a noisy image, the Laplacian is not a denoiser: PSNR drops to 11.99–16.05 dB under Gaussian noise and 15.47–15.73 dB under salt-and-pepper noise, well below the noisy baseline, because the second derivative amplifies high-frequency noise. For Gaussian noise the larger kernels score higher (11.99 $\rightarrow$ 16.05 dB) since their rescaled kernels contain more smoothing. Table 3 shows that sharpening a denoised image also lowers both metrics relative to the denoised image alone (for example, Gaussian 5×5 under Gaussian noise: 28.25 $\rightarrow$ 25.21 dB). This is expected from the evaluation protocol, because the reference is the original *unsharpened* image: any contrast enhancement, even visually pleasing, increases the pixel-wise deviation, and residual noise left by the denoiser is amplified. The smallest degradation occurs for the Median filter under impulse noise (28.63 $\rightarrow$ 27.13 dB), where almost no residual noise remains. Sharpening quality was therefore

	Gaussian noise				Salt-and-pepper			
	PSNR (dB)		SSIM		PSNR (dB)		SSIM	
Denoiser	D	D+S	D	D+S	D	D+S	D	D+S
Mean 5×5	27.04	25.22	0.669	0.495	26.04	23.68	0.610	0.422
Gaussian 5×5, σ=1	28.25	25.21	0.696	0.536	26.33	22.70	0.623	0.492
Median 5×5	26.80	23.48	0.623	0.397	28.63	27.13	0.777	0.744

Table 3: Effect of Laplacian sharpening (3×3, $c = 0.5$) after denoising. D: denoised only; D+S: denoised then sharpened.

Method	**Strengths**	**Weaknesses**
Mean	Simplest and fastest; adequate against mild Gaussian noise at 3×3	Blurs edges uniformly; diffuses impulses instead of removing them; PSNR drops for $k > 3$
Gaussian	Best PSNR and SSIM under Gaussian noise; separable and cheap; smooth, natural appearance	Cannot reject impulses; oversmooths when both k and σ are large
Median	Dominant under salt-and-pepper noise (32.12 dB); preserves edges	Slowest, with cost growing quickly in k; weaker than linear filters under Gaussian noise; rounds small structures for large k
Laplacian sharpening	Enhances edges and local contrast	Amplifies noise; halo artefacts; lowers PSNR/SSIM against the clean reference

Table 4: Strengths and weaknesses of the evaluated methods.

assessed qualitatively (Figures 1–5); no dedicated sharpness metric was computed.

In summary, the choice of filter should follow the noise statistics: a Gaussian filter (preferably 5×5, $\sigma = 1$) for additive Gaussian noise, and a small Median filter (3×3) for impulse noise. Laplacian sharpening should be applied only after effective denoising and with a moderate c (Table 4). These conclusions are drawn from a single image and a single noise realisation per noise type; they are consistent across the three kernel sizes, but no confidence intervals are reported.

3 Part B: 3D Reconstruction

3.1 Methodology

The stereo pipeline consists of three stages: epipolar-geometry analysis, disparity estimation, and depth reconstruction.

Epipolar geometry. Corresponding points $\mathbf{x}$ (left) and $\mathbf{x}'$ (right), in homogeneous pixel coordinates, satisfy the epipolar constraint

$$\mathbf{x}'^{\top} F \, \mathbf{x} = 0, \tag{11}$$

where F is the 3×3 fundamental matrix of rank 2 (7 degrees of freedom), and $\mathbf{l}' = F\mathbf{x}$ is the epipolar line in the right image on which $\mathbf{x}'$ must lie. This reduces the search for a correspondence from two dimensions to one. For a rectified pair the epipolar lines are horizontal scanlines. F is estimated from SIFT matches filtered by Lowe's ratio test and robustly fitted with RANSAC; its quality is measured on the inliers by two distances,

$$d_{\mathrm{S}} = \frac{|\mathbf{x}'^{\top} F \mathbf{x}|}{\sqrt{(F\mathbf{x})_1^2 + (F\mathbf{x})_2^2 + (F^{\top}\mathbf{x}')_1^2 + (F^{\top}\mathbf{x}')_2^2}}, \tag{12}$$

$$d_{\mathrm{ep}} = \tfrac{1}{2}\big(\operatorname{dist}(\mathbf{x}', F\mathbf{x}) + \operatorname{dist}(\mathbf{x}, F^{\top}\mathbf{x}')\big), \qquad \operatorname{dist}(\mathbf{p}, \mathbf{l}) = \frac{|\mathbf{p}^{\top}\mathbf{l}|}{\sqrt{l_1^2 + l_2^2}}, \tag{13}$$

namely the Sampson distance (first-order approximation of the geometric reprojection error) and the symmetric point-to-epipolar-line distance, both in pixels.

Disparity and depth. For a rectified pair with baseline B and focal length f (in pixels), the disparity of a pixel is $d = x_L - x_R$ and the depth follows from similar triangles:

$$Z = \frac{fB}{d}, \qquad X = \frac{(x - c_x)\, Z}{f}, \qquad Y = \frac{(y - c_y)\, Z}{f}. \tag{14}$$

Larger disparity therefore corresponds to a closer point, and depth resolution degrades quadratically with distance because $\partial Z / \partial d = -fB/d^2$.

Block Matching (BM). BM is a local method: for each pixel of the left image it slides a $b \times b$ window W along the same row of the right image and selects the disparity of lowest matching cost, here the Sum of Absolute Differences,

$$d^*(x, y) = \operatorname*{arg\,min}_{0 \le d < D} \sum_{(i,j) \in W} \big| I_L(x + i,\, y + j) - I_R(x + i - d,\, y + j) \big|, \tag{15}$$

where D is `numDisparities`. Each pixel is decided independently, so BM is fast but unreliable in textureless or repetitive regions. A small window preserves boundaries but is noise-sensitive; a large window is robust but blurs depth discontinuities.

Semi-Global Block Matching (SGBM). SGBM approximates the minimisation of a global energy that adds a smoothness prior to the matching cost,

$$E(D) = \sum_{\mathbf{p}} \Big(C(\mathbf{p}, D_{\mathbf{p}}) + \sum_{\mathbf{q} \in \mathcal{N}_{\mathbf{p}}} P_1 \mathbb{1}\big[|D_{\mathbf{p}} - D_{\mathbf{q}}| = 1\big] + P_2 \mathbb{1}\big[|D_{\mathbf{p}} - D_{\mathbf{q}}| > 1\big] \Big), \tag{16}$$

by aggregating costs along several one-dimensional paths $\mathbf{r}$ through the image:

$$\begin{aligned} L_{\mathbf{r}}(\mathbf{p}, d) = C(\mathbf{p}, d) + \min\Big(& L_{\mathbf{r}}(\mathbf{p} - \mathbf{r}, d),\ L_{\mathbf{r}}(\mathbf{p} - \mathbf{r}, d \pm 1) + P_1,\ \min_k L_{\mathbf{r}}(\mathbf{p} - \mathbf{r}, k) + P_2 \Big) \\ & - \min_k L_{\mathbf{r}}(\mathbf{p} - \mathbf{r}, k), \end{aligned} \tag{17}$$

with $S(\mathbf{p}, d) = \sum_{\mathbf{r}} L_{\mathbf{r}}(\mathbf{p}, d)$ and $d^* = \arg\min_d S(\mathbf{p}, d)$. The penalty P_1 tolerates small disparity changes (slanted surfaces), while $P_2 > P_1$ penalises large jumps but still permits genuine depth discontinuities. In OpenCV the per-pixel cost is a sampling-insensitive (Birchfield–Tomasi) measure summed over the block.

Matches	Inliers	Inlier ratio (%)	Mean d_{ep} (px)	Mean d_{S} (px)	Median $\lvert\Delta y\rvert$ (px)
337	315	93.47	0.239	0.169	0.117

Table 5: Fundamental-matrix estimation (SIFT, ratio test 0.75, RANSAC threshold 1.0 px). Errors are averaged over the inliers.

Reprojection. The disparity map is converted to a point cloud with the 4×4 matrix

$$Q = \begin{bmatrix} 1 & 0 & 0 & -c_x \\ 0 & 1 & 0 & -c_y \\ 0 & 0 & 0 & f \\ 0 & 0 & 1/B & 0 \end{bmatrix}, \qquad \begin{bmatrix} X \\ Y \\ Z \\ W \end{bmatrix} = Q \begin{bmatrix} x \\ y \\ d \\ 1 \end{bmatrix}, \tag{18}$$

so that, after dividing by $W = d/B$, $X = (x - c_x)B/d$, $Y = (y - c_y)B/d$, and $Z = fB/d$, in agreement with the depth equation above.

3.2 Implementation and Results

The experiment uses a rectified stereo pair of 450×375 pixels from the Middlebury datasets [6], without ground-truth disparity; consequently accuracy measures such as RMSE or bad-pixel rate cannot be computed, and the comparison relies on the valid-pixel ratio, an epipolar-consistency measure, and runtime. The calibration parameters are not available, so f is set to the image width (450 px) and $B = 1$; the reconstructed geometry is therefore correct up to a global scale and depths are expressed in units of the baseline.

Fundamental matrix. SIFT descriptors are matched with `cv2.BFMatcher` (L2 norm, $k = 2$) and Lowe's ratio test ($\tau = 0.75$). F is estimated with `cv2.findFundamentalMat` (`FM_RANSAC`, threshold 1.0 px, confidence 0.99) and normalised to unit Frobenius norm. Table 5 summarises the estimation: 315 of 337 matches (93.5%) are inliers, the mean errors are below 0.25 px, and the median vertical offset of matched points is 0.117 px, confirming that the pair is effectively rectified. Figure 7 shows epipolar lines for 12 randomly selected inliers; each line passes through the corresponding point in the other image and the lines are almost horizontal.

Disparity maps. Following [2], disparity is computed with `cv2.StereoBM_create` and `cv2.StereoSGBM_create`; OpenCV returns fixed-point values scaled by 16, so the output is divided by 16. For BM the settings are an X-Sobel prefilter (cap 31), texture threshold 10, uniqueness ratio 10, speckle window 100 with range 2, and `disp12MaxDiff` $= 1$ (left–right consistency check). For SGBM, $P_1 = 8b^2$ and $P_2 = 32b^2$ (single-channel images), uniqueness ratio 10, the same speckle and consistency settings, prefilter cap 63, and the three-direction mode `STEREO_SGBM_MODE_SGBM_3WAY`. The search range is $D \in \{64, 128\}$, the BM block size $b \in \{5, 15\}$, and the SGBM block size $b \in \{3, 7\}$. Figure 8 shows the configuration with the highest valid-pixel ratio for each method ($D = 64$; BM with $b = 15$ and SGBM with $b = 7$); black pixels are invalid.

Point cloud. The best SGBM disparity ($D = 64$, $b = 7$) is reprojected with Q using `cv2.reprojectImageTo3D`. Pixels with non-positive disparity or non-finite coordinates

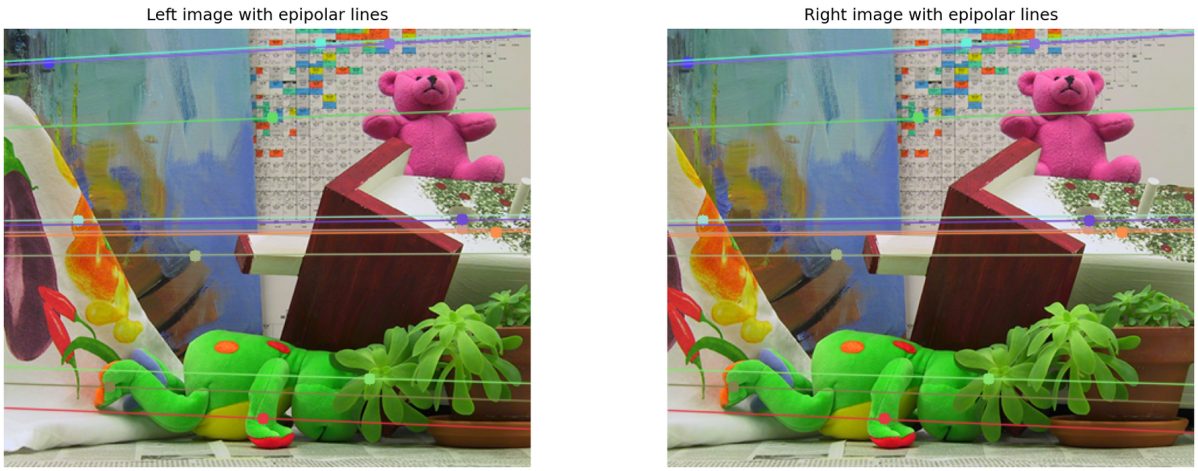


Figure 7: Epipolar lines of 12 inlier correspondences in the left (first) and right (second) image.

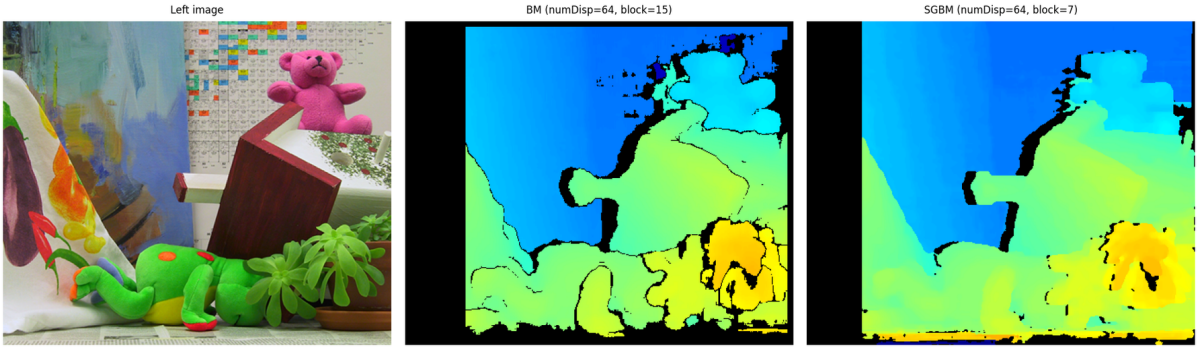


Figure 8: Left image and colour-coded disparity maps of the best BM and SGBM configurations (black: invalid pixels; warmer colour: larger disparity, i.e., closer).

are removed, together with the farthest 1% of points (largest Z), leaving 134,225 coloured points that are exported to a PLY file. Figure 9 shows four views of a random subset of 40,000 points. The front view reproduces the layout of the scene, and the top view reveals the layered depth structure (background wall, mid-ground objects, foreground objects) that is difficult to perceive in the 2D disparity map. Holes appear behind foreground objects, corresponding to occluded regions where no valid disparity exists.

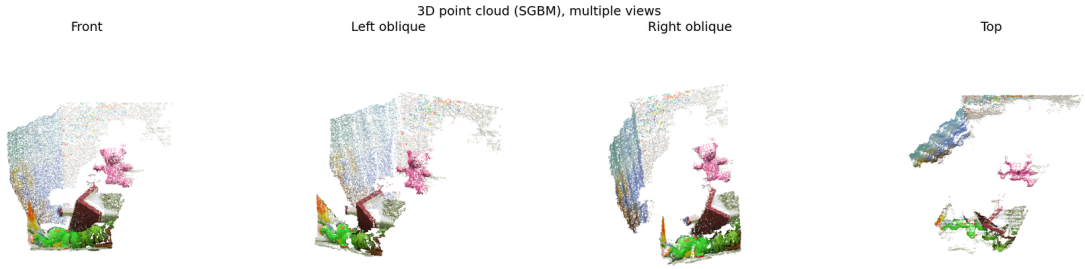


Figure 9: Point cloud reconstructed from the SGBM disparity map, seen from four viewpoints (front, left oblique, right oblique, top).

Method	D	Block size	Valid pixels (%)	Mean d_S (px)	Runtime (ms)
BM	64	5	53.96	0.191	21.47
BM	64	15	69.90	0.188	13.63
SGBM	64	3	81.02	0.187	31.44
SGBM	64	7	**81.17**	0.191	32.18
BM	128	5	39.82	0.188	20.52
BM	128	15	56.55	0.185	24.53
SGBM	128	3	67.03	0.180	42.96
SGBM	128	7	67.03	0.184	46.47

Table 6: BM versus SGBM for different search ranges D (`numDisparities`) and block sizes.

3.3 Comparative Analysis

Table 6 lists the valid-pixel ratio $V = |\{\mathbf{p} : d(\mathbf{p}) > 0\}|/(WH)$, the mean Sampson distance of the disparity-induced correspondences $(\mathbf{x}, \mathbf{x} - (d, 0)^\top)$ over 5,000 random valid pixels, and the runtime (median of three runs) for all eight configurations.

Quantitative comparison. At the same search range, SGBM yields a markedly higher valid ratio than BM: 81.0–81.2% versus 54.0–69.9% for $D = 64$ and 67.0% versus 39.8–56.6% for $D = 128$. The cost is runtime: the best SGBM configuration (32.2 ms) is about 2.4× slower than the best BM one (13.6 ms). For BM the block size has a strong effect (valid ratio 54.0% → 69.9% when b grows from 5 to 15), because larger windows contain more texture and survive the uniqueness and texture tests; SGBM is almost insensitive to b between 3 and 7, since the smoothness aggregation dominates the decision. Doubling D from 64 to 128 lowers the valid ratio by roughly 14 percentage points for SGBM (81.0 → 67.0%) and by 13–14 points for BM. This agrees with the geometry of the search: the first D columns of the left image have no counterpart in the right image, which is $64/450 \approx 14.2\%$ of the width for $D = 64$ and 28.4% for $D = 128$, so the extra range merely enlarges the invalid left border (visible as the black strip in Figure 8) and increases the runtime without capturing additional scene content. The mean Sampson distance is nearly identical for all settings (0.18–0.19 px) and therefore does not discriminate between methods: the correspondences are generated along the same scanline of a rectified pair, so the metric mainly confirms the quality of the estimated F and of the rectification rather than the accuracy of the disparity. A ground-truth comparison would be needed for a statement on absolute accuracy.

Qualitative comparison. The BM map is fragmented, with many small invalid regions and holes along object contours, and it fails to produce a reliable estimate on several smooth surfaces. The SGBM map is visibly smoother and more complete: planar regions such as the back wall and the box surfaces are filled consistently and object contours are continuous. Both methods share two limitations visible in Figure 8: the invalid left border and the black bands adjacent to foreground objects that correspond to half-occlusions. SGBM additionally shows a thin band of erroneous values along the bottom image border. Overall, SGBM provides denser and visually more coherent depth, in agreement with the role of its smoothness prior, while BM remains attractive when computation time is the main constraint.

4 Part C: Image Stitching

4.1 Methodology

The stitching pipeline for four images $I_1, \dots, I_4$ ordered from left to right consists of feature detection and description, matching, robust homography estimation, warping into a common frame, and blending.

Feature detection and description. Three detectors are compared. *SIFT* finds extrema of the difference-of-Gaussians $D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$ over space and scale, assigns a dominant gradient orientation, and describes the neighbourhood by gradient histograms in a 4×4 grid with 8 orientation bins, i.e., a 128-dimensional real-valued descriptor invariant to scale and rotation. *ORB* combines the FAST corner detector, an orientation obtained from the intensity centroid, $\theta = \operatorname{atan2}(m_{01}, m_{10})$, and a rotation-aware binary BRIEF descriptor; it is very fast but its scale coverage is coarser. *AKAZE* builds a non-linear scale space by solving the diffusion equation $\partial L/\partial t = \operatorname{div}\big(c(x, y, t)\nabla L\big)$ with an edge-stopping conductivity $c = g(|\nabla L_\sigma|)$, so that smoothing respects object boundaries, and describes keypoints with a binary M-LDB descriptor. At most 5,000 features per image are retained for SIFT and ORB; AKAZE uses its default threshold.

Matching. Descriptors are matched by brute force, with the L2 norm for the real-valued SIFT descriptor and the Hamming distance (XOR followed by bit counting) for the binary ORB and AKAZE descriptors. For each query descriptor the two nearest neighbours at distances $d_1 \le d_2$ are found, and the match is kept only if Lowe's ratio test holds,

$$\frac{d_1}{d_2} < \tau, \qquad \tau = 0.75, \tag{19}$$

which rejects ambiguous matches whose best candidate is not clearly better than the runner-up.

Homography estimation. Two views of a planar scene, or of any scene under a pure camera rotation, are related by a homography $H \in \mathbb{R}^{3\times 3}$ defined up to scale (eight degrees of freedom):

$$\tilde{\mathbf{x}}' \sim H\tilde{\mathbf{x}}, \qquad x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}, \quad y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}. \tag{20}$$

Each correspondence gives two linear equations, so the Direct Linear Transform needs at least $s = 4$ non-collinear pairs. Because matches contain outliers, RANSAC repeatedly fits H to random minimal samples and keeps the model with the largest consensus set. A match is an inlier if its reprojection error is below a threshold τ_r,

$$e_i = \big\|\mathbf{x}'_i - \pi(H\tilde{\mathbf{x}}_i)\big\| < \tau_r, \qquad \pi(X, Y, W) = \left(\tfrac{X}{W}, \tfrac{Y}{W}\right), \tag{21}$$

and the number of iterations needed to draw at least one outlier-free sample with confidence p is

$$N \ge \frac{\log(1-p)}{\log\big(1 - w^s\big)}, \tag{22}$$

where w is the inlier ratio. In the experiments $p = 0.995$, at most 5,000 iterations are allowed, and $\tau_r \in \{1, 3, 5\}$ px is studied (with $\tau_r = 3$ px for the final panoramas).

Estimation quality is summarised by the root-mean-square reprojection error over the inliers,

$$\mathrm{RMSE} = \sqrt{\frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} e_i^2}. \tag{23}$$

Warping to a common frame. Let $H^{(i)}$ denote the homography mapping I_i to I_{i+1}. The second image I_2 is chosen as the reference, so that errors do not accumulate in one direction and the outer images are warped over fewer composed transformations. The transformations to the reference frame are

$$H_{1\to 2} = H^{(1)}, \qquad H_{3\to 2} = \left(H^{(2)}\right)^{-1}, \qquad H_{4\to 2} = H_{3\to 2}\left(H^{(3)}\right)^{-1}, \tag{24}$$

each normalised by its element $(3,3)$. The corners of every image are transformed to find the bounding box $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ of the panorama, and a translation (with a scale factor $s \le 1$ that limits the canvas to 6,000 px) places it in positive coordinates:

$$T = \begin{bmatrix} s & 0 & -s\,x_{\min} \\ 0 & s & -s\,y_{\min} \\ 0 & 0 & 1 \end{bmatrix}, \qquad M_i = T\,H_{i\to 2}. \tag{25}$$

Each image is resampled with inverse mapping and bilinear interpolation (`cv2.warpPerspective`), i.e., every canvas pixel looks up its source location through M_i^{-1}, which avoids holes.

Blending. In the main experiments the images are combined by *feathering*: the weight of image i at a canvas pixel $\mathbf{p}$ is the Euclidean distance transform of its warped validity mask m_i (the distance to the nearest invalid pixel), so that weights decrease towards image borders,

$$I_{\mathrm{pano}}(\mathbf{p}) = \frac{\sum_i w_i(\mathbf{p})\, I_i(\mathbf{p})}{\sum_i w_i(\mathbf{p})}, \qquad w_i = \mathrm{DT}(m_i). \tag{26}$$

As an additional experiment, *multi-band blending* is applied with 5 levels: each warped image is decomposed into a Laplacian pyramid $\mathcal{L}_\ell^{(i)} = \mathcal{G}_\ell^{(i)} - \mathrm{up}\left(\mathcal{G}_{\ell+1}^{(i)}\right)$, the hard assignment masks (the image with the largest weight at each pixel) are smoothed by a Gaussian pyramid $\mathcal{M}_\ell^{(i)}$, and each frequency band is blended as $B_\ell = \sum_i \mathcal{L}_\ell^{(i)}\mathcal{M}_\ell^{(i)} / \sum_i \mathcal{M}_\ell^{(i)}$ before the pyramid is collapsed. Low frequencies are therefore mixed over a wide transition zone, whereas high frequencies are mixed over a narrow one.

4.2 Implementation and Results

The four input photographs, taken from a public image-stitching dataset collection [4] and ordered from left to right with overlapping fields of view, are resized to a maximum width of 1,000 pixels. SIFT, ORB, and AKAZE are created with `cv2.SIFT_create(5000)`, `cv2.ORB_create(5000)`, and `cv2.AKAZE_create()`, matched with `cv2.BFMatcher`, and following [3], the homographies are estimated with `cv2.findHomography` (`cv2.RANSAC`). Figure 10 shows the detected keypoints in the first image and 60 randomly chosen inlier matches for the pair I_1–I_2: for $\tau_r = 3$ px, SIFT retains 551 inliers out of 659 ratio-test matches, ORB 470 of 630, and AKAZE 126 of 156. The ORB keypoints, shown with their

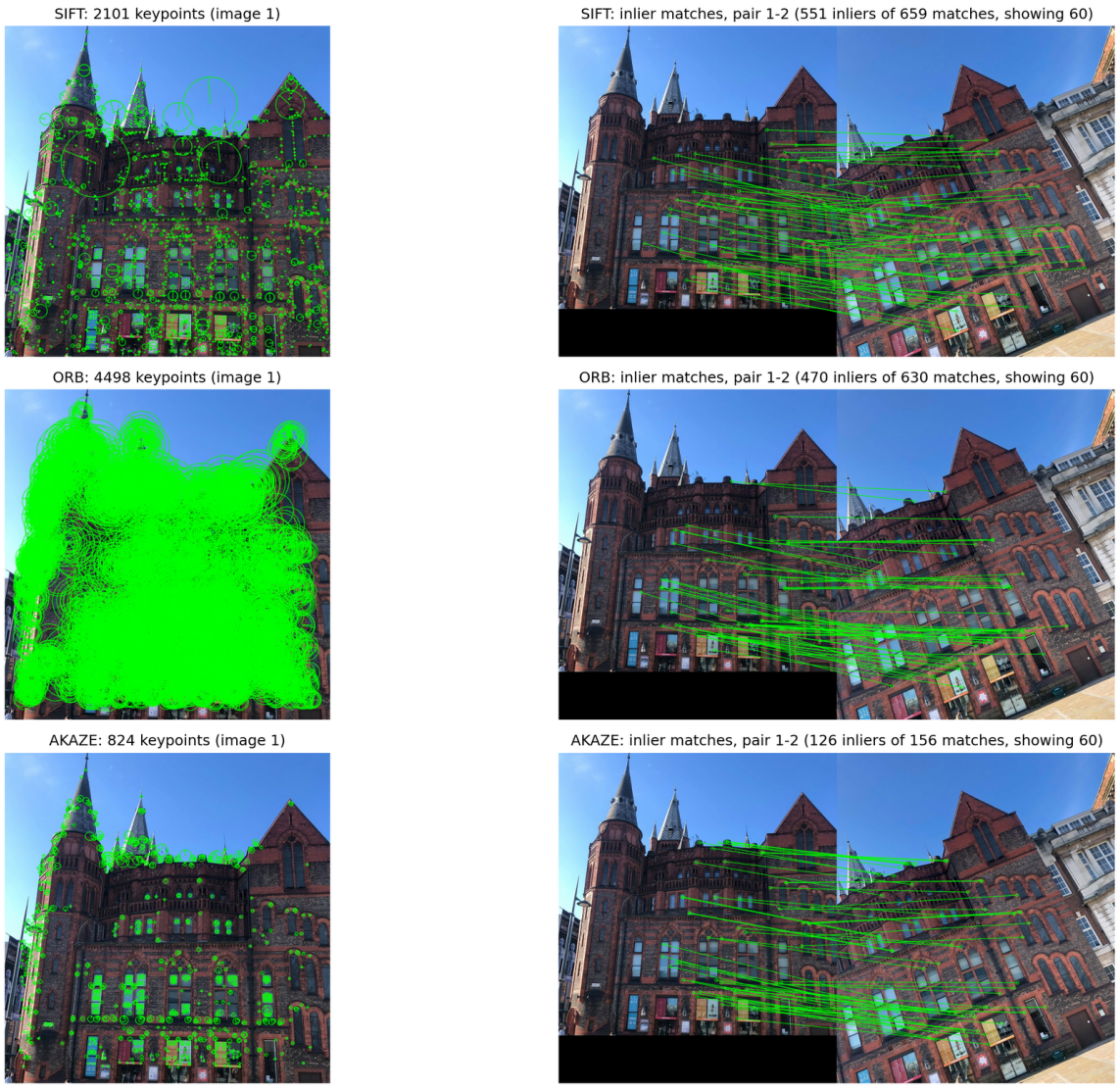


Figure 10: Left: keypoints detected in the first image. Right: 60 inlier matches between the first and second image ($\tau_r = 3$ px) for SIFT, ORB, and AKAZE (top to bottom).

Detector	τ_r (px)	Initial matches	Inliers	Inlier ratio (%)	RMSE (px)
SIFT	1.0	2062	1074	51.97	0.568
SIFT	3.0	2062	1766	85.48	0.966
SIFT	5.0	2062	1856	89.78	1.160
ORB	1.0	2255	746	30.92	0.644
ORB	3.0	2255	1686	73.61	1.392
ORB	5.0	2255	1896	83.44	1.764
AKAZE	1.0	1261	617	46.72	0.527
AKAZE	3.0	1261	1056	83.25	0.977
AKAZE	5.0	1261	1155	91.52	1.242

Table 7: Influence of the RANSAC threshold τ_r. Matches and inliers are summed over the three adjacent pairs; the inlier ratio and RMSE are averaged over the pairs.

scale circles, cover the whole image densely, whereas SIFT and AKAZE concentrate on structured regions such as windows, towers, and roof edges.

Figure 11 shows the panoramas obtained with the three detectors and feathering, and Figure 12 shows the SIFT result in more detail. All three detectors produce geometrically consistent results: the façade lines and window rows continue across the overlap regions and no obvious ghosting is visible at this scale. The overall shape of the mosaic is tilted and has an irregular border because the reference image is itself rotated relative to the horizontal and because the panorama is not cropped; the black areas are canvas pixels not covered by any image. A mild brightness difference between neighbouring images remains visible in the sky region.

4.3 Comparative Analysis

RANSAC threshold. Table 7 reports, for each detector and threshold τ_r, the number of inliers summed over the three adjacent pairs, the mean per-pair inlier ratio, and the mean per-pair RMSE. Increasing τ_r from 1 to 5 px consistently increases the number of inliers and the inlier ratio (SIFT: 52.0% $\rightarrow$ 85.5% $\rightarrow$ 89.8%) but also the reprojection error (SIFT: 0.568 $\rightarrow$ 0.966 $\rightarrow$ 1.160 px). The two trends are not independent: the RMSE is computed over the inliers only and is therefore bounded by τ_r, so a lower threshold lowers the RMSE almost by construction, while a higher threshold admits geometrically less consistent matches, which are harmful in the presence of parallax or lens distortion. The step from 1 to 3 px recovers a large number of correct matches (SIFT: 1074 $\rightarrow$ 1766 inliers), whereas the step from 3 to 5 px adds only 90 inliers and increases the RMSE by about 0.19 px, which justifies $\tau_r = 3$ px as the working value. ORB is the most threshold-sensitive detector (inlier ratio 30.9% $\rightarrow$ 73.6% $\rightarrow$ 83.4%), which suggests that its keypoints are localised less precisely.

Detector comparison. Table 8 compares the three detectors at $\tau_r = 3$ px, with the inlier ratio now pooled over all pairs (inliers/initial matches), which explains the small difference from the per-pair average in Table 7 (e.g., SIFT 85.65% versus 85.48%). The total time covers extraction, matching, RANSAC, warping, and blending, and is a single



Figure 11: Panoramas obtained with SIFT, ORB, and AKAZE (top to bottom) using feathering and $\tau_r = 3$ px.



Figure 12: Final panorama obtained with SIFT, RANSAC, and feathering (not cropped).

Detector	Blending	Keypoints	Matches	Inliers	Ratio (%)	RMSE (px)
SIFT	feather	11004	2062	**1766**	**85.65**	**0.966**
ORB	feather	19306	2255	1686	74.77	1.392
AKAZE	feather	6065	1261	1056	83.74	0.977
SIFT	multi-band	11004	2062	1766	85.65	0.966

Detector	Blending	Total time (ms)
SIFT	feather	1813
ORB	feather	1506
AKAZE	feather	**815**
SIFT	multi-band	2009

Table 8: Detector comparison at $\tau_r = 3$ px. Keypoints are summed over the four images, matches and inliers over the three pairs.

measurement per configuration.

SIFT achieves the best geometric quality: the most inliers (1766), the highest inlier ratio (85.65%), and the lowest RMSE (0.966 px). ORB detects the most keypoints (19,306) and returns the most initial matches (2255), but only 74.8% of them are consistent with a homography and its RMSE is about 44% higher (1.392 px); a larger number of keypoints thus does not imply better matches. AKAZE, with only about 55% of SIFT's keypoints,

reaches nearly the same inlier ratio (83.74%) and RMSE (0.977 px) and is the fastest pipeline (815 ms, less than half of SIFT's 1813 ms), which makes it an attractive compromise when robustness and speed are both required. ORB is faster than SIFT overall (1506 versus 1813 ms), although the difference is moderate because warping and blending are identical for all detectors. In terms of the resulting panoramas (Figure 11), the three detectors are visually very similar for this scene, which has abundant texture and a small viewpoint change; their differences are therefore more evident in the numerical indicators than in the images.

Blending. Figure 13 compares feathering with multi-band blending for the SIFT result. In the enlarged region, building details are indistinguishable between the two methods, while the multi-band result shows a slightly smoother transition of the sky tone across the overlap. The improvement is modest for this scene because the exposure difference between the images is small, and it costs about 10.8% more time (2009 versus 1813 ms). Multi-band blending is expected to be more beneficial when the exposure differences or the amount of fine detail in the overlap are larger.

Limitations. A single homography per image pair assumes a planar or distant scene, or a purely rotating camera. The scene here contains façades at different depths, so small residual misalignments are possible; furthermore, errors in the chained homographies $H_{4\to 2}$ accumulate with the number of composed transformations, and no global refinement (bundle adjustment) is performed. No ground truth exists for the panorama, so the RMSE over inliers and visual inspection are the only available quality indicators.

5 Conclusion

This project implemented and compared traditional methods for three vision tasks. In image filtering, the best method depends on the noise statistics: a 5×5 Gaussian filter with $\sigma = 1$ was the best compromise for Gaussian noise (28.25 dB, 0.292 ms), a 3×3 Median filter was far superior for salt-and-pepper noise (32.12 dB versus at most 26.42 dB for linear filters), and Laplacian sharpening enhanced edges but amplified noise and reduced PSNR and SSIM against the clean reference. In stereo reconstruction, SGBM produced denser and visually smoother disparity maps than BM (81.2% versus 69.9% valid pixels) at about $2.4\times$ the runtime, and the point cloud reproduced the scene layout up to scale. In stitching, SIFT gave the highest inlier ratio and lowest reprojection error, AKAZE offered nearly the same quality in less than half the time, and ORB, despite producing the most keypoints, had the lowest inlier ratio.

The conclusions are subject to several limitations: each task used a single image or scene and one noise realisation, the stereo pair had no ground truth so that disparity accuracy could not be measured, and the panorama relies on a single homography per pair without global refinement or cropping. Future work could add ground-truth evaluation and left–right-consistency post-filtering (or weighted-least-squares filtering) for stereo, edge-preserving denoisers such as the bilateral filter, and cylindrical warping, bundle adjustment, and seam optimisation for stitching. Combining the depth information of Part B with the processing of Parts A and C is a further direction to handle scenes with strong parallax.



Figure 13: Enlarged region of the SIFT panorama: feathering (left) versus multi-band blending (right).

6 References

[1] OpenCV Documentation, *Image Filtering*, [Online]. Available: https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html

[2] OpenCV Documentation, *Stereo Correspondence*, [Online]. Available: https://docs.opencv.org/4.x/dd/d53/tutorial_py_depthmap.html

[3] OpenCV Documentation, *Feature Matching + Homography to find Objects*, [Online]. Available: https://docs.opencv.org/4.x/d1/de0/tutorial_py_feature_homography.html

[4] preethamam, *Image-Stitching-Datasets-Compilation-Collection: A compilation of image stitching datasets from USC and other open source providers*, GitHub, [Online]. Available: https://github.com/preethamam/Image-Stitching-Datasets-Compilation-Collection

[5] Wikimedia Commons, *File:Albert Einstein sticks his tongue.jpg*, [Online]. Available: https://commons.wikimedia.org/wiki/File:Albert_Einstein_sticks_his_tongue.jpg

[6] Middlebury Computer Vision, *Datasets*, [Online]. Available: https://vision.middlebury.edu/flow/data/

A Appendix: Python Code

The listings contain the core processing code of each part; plotting and file-export code is omitted (the complete notebook is part of the submission).

A.1 Part A: Image Filtering

```
import os, re, time, math
import cv2
import numpy as np
import pandas as pd
from skimage.metrics import structural_similarity as _ssim

# HELPERS
def read_img(path, flags=cv2.IMREAD_COLOR):
    im = cv2.imread(path, flags)
    if im is None:
        raise FileNotFoundError(f"Cannot read '{path}'. Please upload the image and fix CONFIG.")
    return im

def resize_max_side(img, max_side):
    if not max_side:
        return img, 1.0
    s = min(1.0, max_side / max(img.shape[:2]))
    if s < 1.0:
        img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
    return img, s
```

```
21
22 def rgb(img):
23     return cv2.cvtColor(img, cv2.COLOR_BGR2RGB) if img.ndim == 3 else
           img
24
25 def savefig(fig, path):
26     fig.savefig(path, dpi=150, bbox_inches="tight")
27
28 def timeit(fn, repeat=5):
29     """Run fn several times and return (result, median time in ms)."""
30     ts, out = [], None
31     for _ in range(repeat):
32         t0 = time.perf_counter()
33         out = fn()
34         ts.append((time.perf_counter() - t0) * 1000)
35     return out, float(np.median(ts))
36
37 def show_grid(images, titles, ncols, path=None, suptitle=None,
       cell=3.2):
38     n = len(images)
39     nrows = int(math.ceil(n / ncols))
40     fig, axes = plt.subplots(nrows, ncols, figsize=(cell * ncols, cell
           * nrows))
41     axes = np.atleast_1d(axes).ravel()
42     for ax in axes:
43         ax.axis("off")
44     for ax, im, t in zip(axes, images, titles):
45         ax.imshow(rgb(im) if im.ndim == 3 else im, cmap="gray", vmin=0,
               vmax=255)
46         ax.set_title(t, fontsize=8)
47     if suptitle:
48         fig.suptitle(suptitle, fontsize=11)
49     fig.tight_layout()
50     if path:
51         savefig(fig, path)
52     plt.show()
53
54 # A1. Load the image and add noise
55 img_a_color, _ = resize_max_side(read_img(PATH_A), A_MAX_SIDE)
56 clean = cv2.cvtColor(img_a_color, cv2.COLOR_BGR2GRAY)
57 rng = np.random.default_rng(SEED)
58
59 def add_gaussian_noise(img, sigma=25):
60     return np.clip(img.astype(np.float64) + rng.normal(0, sigma,
           img.shape), 0, 255).astype(np.uint8)
61
62 def add_salt_pepper(img, amount=0.05):
63     out = img.copy()
64     r = rng.random(img.shape)
65     out[r < amount / 2] = 0
66     out[r > 1 - amount / 2] = 255
67     return out
68
69 GAUSS_SIGMA, SP_AMOUNT = 25, 0.05
70 NOISE_G = f"Gaussian noise (sigma={GAUSS_SIGMA})"
71 NOISE_SP = f"Salt&Pepper noise (p={SP_AMOUNT})"
72 noisy = {NOISE_G: add_gaussian_noise(clean, GAUSS_SIGMA), NOISE_SP:
       add_salt_pepper(clean, SP_AMOUNT)}
```

```
73
74 # A2. Filters and metrics
75 KS = [3, 5, 7]
76 SIGMAS = [1.0, 2.0]
77 LAP_C = [0.5]
78
79 def f_mean(im, k):
80     return cv2.blur(im, (k, k))
81
82 def f_gauss(im, k, s):
83     return cv2.GaussianBlur(im, (k, k), s)
84
85 def f_median(im, k):
86     return cv2.medianBlur(im, k)
87
88 def _lap_scale(k):
89     """Normalize the Laplacian of size k so that the sum of
        |coefficients| equals that of the standard 3x3 kernel (=8),
90     so that sharpening strength is comparable across kernel sizes."""
91     imp = np.zeros((41, 41), np.float32)
92     imp[20, 20] = 1
93     kern = cv2.Laplacian(imp, cv2.CV_32F, ksize=k)
94     return 8.0 / float(np.abs(kern).sum())
95
96 def f_lap_sharpen(im, k, c):
97     lap = cv2.Laplacian(im.astype(np.float32), cv2.CV_32F, ksize=k,
        scale=_lap_scale(k))
98     return np.clip(im.astype(np.float32) - c * lap, 0,
        255).astype(np.uint8)
99
100 def metrics(ref, out):
101     mse = float(np.mean((ref.astype(np.float64) -
        out.astype(np.float64)) ** 2))
102     psnr = float("inf") if mse == 0 else 10 * np.log10(255.0 ** 2 / mse)
103     ssim = float(_ssim(ref, out, data_range=255))
104     return mse, psnr, ssim
105
106 DENOISE_CFGS, SHARPEN_CFGS = [], []
107 for k in KS:
108     DENOISE_CFGS.append(("Mean", f"k={k}", lambda im, k=k: f_mean(im,
        k)))
109 for k in KS:
110     for s in SIGMAS:
111         DENOISE_CFGS.append(("Gaussian", f"k={k},sigma={s}", lambda im,
            k=k, s=s: f_gauss(im, k, s)))
112 for k in KS:
113     DENOISE_CFGS.append(("Median", f"k={k}", lambda im, k=k:
        f_median(im, k)))
114 for k in KS:
115     for c in LAP_C:
116         SHARPEN_CFGS.append(("Laplacian sharpen", f"k={k},c={c}",
            lambda im, k=k, c=c: f_lap_sharpen(im, k, c)))
117
118 rowsA, outsA = [], {}
119 def _record(noise, method, params, out, ms):
120     mse, psnr, ssim = metrics(clean, out)
121     rowsA.append(dict(Noise=noise, Method=method, Params=params,
        PSNR_dB=psnr, SSIM=ssim, MSE=mse, Time_ms=ms))
```

```
    outsA[(noise, method, params)] = out

for nz, im in noisy.items():
    _record(nz, "No filter", "-", im, 0.0)
    for m, p, fn in DENOISE_CFGS + SHARPEN_CFGS:
        out, ms = timeit(lambda fn=fn, im=im: fn(im))
        _record(nz, m, p, out, ms)

# fixed reference settings, identical for both noise types (used by the visual grids and the Denoise->Sharpen pipeline)
GRID_K, GRID_SIGMA = 5, 1.0
GRID_CFG = {"Mean": f"k={GRID_K}", "Gaussian": f"k={GRID_K},sigma={GRID_SIGMA}", "Median": f"k={GRID_K}"}
PIPE_K, PIPE_C = 3, LAP_C[0]
GRID_DENOISE = {"Mean": lambda im: f_mean(im, GRID_K),
                "Gaussian": lambda im: f_gauss(im, GRID_K, GRID_SIGMA),
                "Median": lambda im: f_median(im, GRID_K)}
for nz in noisy:
    for m, fn in GRID_DENOISE.items():
        out, ms = timeit(lambda fn=fn, nz=nz: f_lap_sharpen(fn(noisy[nz]), PIPE_K, PIPE_C))
        _record(nz, "Denoise->Sharpen", f"{m} {GRID_CFG[m]} + Lap k={PIPE_K},c={PIPE_C}", out, ms)

dfA = pd.DataFrame(rowsA).round({"PSNR_dB": 2, "SSIM": 4, "MSE": 2, "Time_ms": 3})
dfA.to_csv(f"{OUT}/tables/partA_results.csv", index=False)
```

A.2 Part B: 3D Reconstruction

```
import time
import cv2
import numpy as np
import pandas as pd
# gL, gR: grayscale left/right images; L_color: left color image; wB, hB: image size

# B2. Fundamental matrix (RANSAC) and epipolar lines
def hom(p):
    return np.hstack([p, np.ones((len(p), 1))])

def sampson_dist(F, p1, p2):
    """Sampson distance (px) for each point pair."""
    x1, x2 = hom(p1), hom(p2)
    Fx1, Ftx2 = x1 @ F.T, x2 @ F
    num = np.sum(x2 * Fx1, axis=1) ** 2
    den = Fx1[:, 0] ** 2 + Fx1[:, 1] ** 2 + Ftx2[:, 0] ** 2 + Ftx2[:, 1] ** 2
    return np.sqrt(num / np.maximum(den, 1e-12))

def epipolar_dist(F, p1, p2):
    """Symmetric point-to-epipolar-line distance (px)."""
    x1, x2 = hom(p1), hom(p2)
    l2, l1 = x1 @ F.T, x2 @ F
    d2 = np.abs(np.sum(x2 * l2, axis=1)) / np.maximum(np.hypot(l2[:, 0], l2[:, 1]), 1e-12)
    d1 = np.abs(np.sum(x1 * l1, axis=1)) / np.maximum(np.hypot(l1[:, 0], l1[:, 1]), 1e-12)
```

```
    return 0.5 * (d1 + d2)

def estimate_F(g1, g2, ratio=0.75, thr=1.0, conf=0.99):
    sift = cv2.SIFT_create()
    k1, d1 = sift.detectAndCompute(g1, None); k2, d2 =
        sift.detectAndCompute(g2, None)
    knn = cv2.BFMatcher(cv2.NORM_L2).knnMatch(d1, d2, k=2)
    good = [p[0] for p in knn if len(p) == 2 and p[0].distance < ratio
        * p[1].distance]
    p1 = np.float32([k1[m.queryIdx].pt for m in good]); p2 =
        np.float32([k2[m.trainIdx].pt for m in good])
    F, mask = cv2.findFundamentalMat(p1, p2, cv2.FM_RANSAC, thr, conf)
    if F is None or mask is None:
        raise RuntimeError("Could not estimate F (too few matches).
            Check the image pair.")
    F = F[:3] if F.shape[0] > 3 else F                    # OpenCV may
        return several solutions
    F = F / np.linalg.norm(F)                             # normalize the
        scale (F is defined up to a constant)
    inl = mask.ravel() == 1
    return dict(F=F, p1=p1, p2=p2, inl=inl, n_good=len(good))

def draw_epilines(img, lines, pts, seed=0):
    img = img.copy(); h, w = img.shape[:2]; r =
        np.random.default_rng(seed)
    for (a, b, c), p in zip(lines, pts):
        col = tuple(int(x) for x in r.integers(50, 255, 3))
        if abs(b) > 1e-9:
            cv2.line(img, (0, int(-c / b)), (w, int(-(c + a * w) / b)),
                col, 1, cv2.LINE_AA)
        cv2.circle(img, (int(p[0]), int(p[1])), 5, col, -1)
    return img

def epipolar_report(est, tag):
    F, inl = est["F"], est["inl"]
    p1, p2 = est["p1"][inl], est["p2"][inl]
    return dict(Stage=tag, Matches=est["n_good"],
        RANSAC_inliers=int(inl.sum()), Inlier_ratio_pct=100 * inl.mean(),
                Mean_epipolar_err_px=float(epipolar_dist(F, p1,
                    p2).mean()),
                Mean_Sampson_err_px=float(sampson_dist(F, p1,
                    p2).mean()),
                Median_vertical_shift_px=float(np.median(np.abs(p1[:,
                    1] - p2[:, 1]))))

estF = estimate_F(gL, gR)
repF = [epipolar_report(estF, "original pair")]

rs = np.random.default_rng(0)
inl_idx = np.flatnonzero(estF["inl"])
sel = rs.choice(inl_idx, size=min(12, len(inl_idx)), replace=False)
pL, pR = estF["p1"][sel], estF["p2"][sel]
linesR = cv2.computeCorrespondEpilines(pL.reshape(-1, 1, 2), 1,
    estF["F"]).reshape(-1, 3)
linesL = cv2.computeCorrespondEpilines(pR.reshape(-1, 1, 2), 2,
    estF["F"]).reshape(-1, 3)
epiL = draw_epilines(L_color, linesL, pL, seed=1); epiR =
    draw_epilines(R_color, linesR, pR, seed=1)
```

```
fig, ax = plt.subplots(1, 2, figsize=(14, 5))
ax[0].imshow(rgb(epiL)); ax[0].set_title("Left image with epipolar lines")
ax[1].imshow(rgb(epiR)); ax[1].set_title("Right image with epipolar lines")
for a in ax: a.axis("off")
fig.tight_layout(); savefig(fig, f"{OUT}/partB/B_epipolar_lines.png"); plt.show()

# optional: rectify using F if the pair is not rectified
F_use, gL_use, gR_use, L_use = estF["F"], gL, gR, L_color
if RECTIFY_UNCALIBRATED:
    p1i, p2i = estF["p1"][estF["inl"]], estF["p2"][estF["inl"]]
    ok, H1, H2 = cv2.stereoRectifyUncalibrated(p1i, p2i, estF["F"], (wB, hB))
    if ok:
        gL_use = cv2.warpPerspective(gL, H1, (wB, hB)); gR_use = cv2.warpPerspective(gR, H2, (wB, hB))
        L_use = cv2.warpPerspective(L_color, H1, (wB, hB))
        est_r = estimate_F(gL_use, gR_use); F_use = est_r["F"]
        repF.append(epipolar_report(est_r, "after uncalibrated rectification"))
        if gt is not None:
            print("WARNING: GT no longer matches after rectification -> RMSE/Bad-pixel are not reliable.")
dfF = pd.DataFrame(repF).round(3)
display(dfF); dfF.to_csv(f"{OUT}/tables/partB_fundamental_matrix.csv", index=False)
if repF[0]["Median_vertical_shift_px"] > 2 and not RECTIFY_UNCALIBRATED:
    print("Note: median vertical shift > 2px -> the pair seems not rectified, try RECTIFY_UNCALIBRATED=True.")

# B3. Disparity: BM vs SGBM with different blockSize and numDisparities
def run_bm(a, b, nd, bs):
    bm = cv2.StereoBM_create(numDisparities=nd, blockSize=bs)
    bm.setPreFilterType(cv2.STEREO_BM_PREFILTER_XSOBEL); bm.setPreFilterCap(31)
    bm.setTextureThreshold(10); bm.setUniquenessRatio(10)
    bm.setSpeckleWindowSize(100); bm.setSpeckleRange(2); bm.setDisp12MaxDiff(1)
    return bm.compute(a, b).astype(np.float32) / 16.0

def run_sgbm(a, b, nd, bs):
    sg = cv2.StereoSGBM_create(minDisparity=0, numDisparities=nd, blockSize=bs,
                               P1=8 * bs * bs, P2=32 * bs * bs, disp12MaxDiff=1, uniquenessRatio=10,
                               speckleWindowSize=100, speckleRange=2, preFilterCap=63,
                               mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY)
    return sg.compute(a, b).astype(np.float32) / 16.0

def colorize(disp, nd):
    d = np.clip(disp / nd, 0, 1)
    c = cv2.applyColorMap((d * 255).astype(np.uint8), cv2.COLORMAP_JET)
    c[disp <= 0] = 0
    return c

```

```
ND_LIST = [64, 128]
BM_BS = [5, 15]
SGBM_BS = [3, 7]

rowsB, dispB = [], {}
rs2 = np.random.default_rng(1)
def eval_disp(disp, method, nd, bs, ms):
    valid = disp > 0
    row = dict(Method=method, numDisparities=nd, blockSize=bs,
       Valid_disparity_pct=100 * valid.mean())
    ys, xs = np.nonzero(valid)
    if len(xs) > 0:
        idx = rs2.choice(len(xs), size=min(5000, len(xs)),
           replace=False)
        p1 = np.stack([xs[idx], ys[idx]], 1).astype(np.float64)
        p2 = p1.copy(); p2[:, 0] -= disp[ys[idx], xs[idx]]
        row["Mean_Sampson_err_px"] = float(sampson_dist(F_use, p1,
           p2).mean())
    else:
        row["Mean_Sampson_err_px"] = np.nan
    if gt is not None:
        m = (gt > 0) & valid
        if m.any():
            e = np.abs(disp[m] - gt[m])
            row.update(RMSE_px=float(np.sqrt(np.mean(e ** 2))),
               Bad2_pct=100 * float((e > 2).mean()))
    row["Runtime_ms"] = ms
    return row

for nd in ND_LIST:
    for bs in BM_BS:
        d, ms = timeit(lambda nd=nd, bs=bs: run_bm(gL_use, gR_use, nd,
           bs), repeat=3)
        dispB[("BM", nd, bs)] = d; rowsB.append(eval_disp(d, "BM", nd,
           bs, ms))
    for bs in SGBM_BS:
        d, ms = timeit(lambda nd=nd, bs=bs: run_sgbm(gL_use, gR_use,
           nd, bs), repeat=3)
        dispB[("SGBM", nd, bs)] = d; rowsB.append(eval_disp(d, "SGBM",
           nd, bs, ms))

dfB = pd.DataFrame(rowsB).round(3)
dfB.to_csv(f"{OUT}/tables/partB_disparity_results.csv", index=False)
display(dfB)

# B5. 3D point cloud from the best SGBM disparity
disp3d = dispB[kSG].copy()
f_px = FOCAL_PX if FOCAL_PX else float(wB)
cx, cy = wB / 2.0, hB / 2.0
Q = np.float32([[1, 0, 0, -cx], [0, 1, 0, -cy], [0, 0, 0, f_px], [0, 0,
    1.0 / BASELINE, 0]])
pts3d = cv2.reprojectImageTo3D(disp3d, Q)             # X=(x-cx)B/d,
    Y=(y-cy)B/d, Z=fB/d
mask = (disp3d > 0) & np.isfinite(pts3d).all(axis=2)
P = pts3d[mask]; C = L_use[mask][:, ::-1]             # BGR -> RGB
zhi = np.percentile(P[:, 2], 99); keep = P[:, 2] < zhi
P, C = P[keep], C[keep]
print(f"Point cloud: {len(P):,} points")
```

```

def save_ply(path, P, C):
    with open(path, "w") as f:
        f.write("ply\nformat ascii 1.0\n" f"element vertex {len(P)}\n"
                "property float x\nproperty float y\nproperty float z\n"
                "property uchar red\nproperty uchar green\nproperty
                   uchar blue\nend_header\n")
        for p, c in zip(P, C):
            f.write(f"{p[0]:.4f} {p[1]:.4f} {p[2]:.4f} {int(c[0])}
                {int(c[1])} {int(c[2])}\n")
save_ply(f"{OUT}/partB/pointcloud_sgbm.ply", P, C)   # open with
    MeshLab/CloudCompare/Open3D

sub = np.random.default_rng(0).choice(len(P), size=min(40000, len(P)),
    replace=False)
Ps, Cs = P[sub], C[sub] / 255.0
views = [(10, -90, "Front"), (20, -60, "Left oblique"), (20, -120,
    "Right oblique"), (85, -90, "Top")]
fig = plt.figure(figsize=(16, 4.5))
for i, (el, az, name) in enumerate(views):
    ax = fig.add_subplot(1, 4, i + 1, projection="3d")
    ax.scatter(Ps[:, 0], Ps[:, 2], -Ps[:, 1], c=Cs, s=0.6, linewidths=0)
    ax.view_init(elev=el, azim=az); ax.set_title(name)
    try:
        ax.set_box_aspect(np.ptp(np.c_[Ps[:, 0], Ps[:, 2], -Ps[:, 1]],
            axis=0))
    except Exception:
        pass
    ax.set_axis_off()
fig.suptitle("3D point cloud (SGBM), multiple views");
    fig.tight_layout()
savefig(fig, f"{OUT}/partB/B_pointcloud_views.png"); plt.show()
```

A.3 Part C: Image Stitching

```
import time, math
import cv2
import numpy as np
import pandas as pd
# imgsC: list of 4 BGR images (left to right); grays: grayscale
    versions; N_IMG = 4; REF_IDX = 1

# C1. Detectors, matcher and homography
DETECTORS = ["SIFT", "ORB", "AKAZE"]          # the assignment needs at
    least 2
RATIO = 0.75
RANSAC_THRS = [1.0, 3.0, 5.0]
STITCH_THR = 3.0                              # RANSAC threshold used
    for the final panoramas

def make_det(name):
    if name == "SIFT":  return cv2.SIFT_create(nfeatures=5000),
        cv2.NORM_L2
    if name == "ORB":   return cv2.ORB_create(nfeatures=5000),
        cv2.NORM_HAMMING
    if name == "AKAZE":
        ctor = getattr(cv2, "AKAZE_create", None) or cv2.AKAZE.create
```

```
        return ctor(), cv2.NORM_HAMMING
    raise ValueError(name)

# drop detectors that are not available in this OpenCV build (the assignment needs at least 2)
_ok = []
for _n in DETECTORS:
    try:
        make_det(_n); _ok.append(_n)
    except Exception as _e:
        print(f"Skipping {_n}: {_e}")
DETECTORS = _ok
assert len(DETECTORS) >= 2, "At least 2 detectors are required (SIFT, ORB)."

def extract(name, grays):
    det, _ = make_det(name)
    t0 = time.perf_counter()
    feats = [det.detectAndCompute(g, None) for g in grays]
    return feats, (time.perf_counter() - t0) * 1000

def match_pair(name, des1, des2):
    _, norm = make_det(name)
    knn = cv2.BFMatcher(norm).knnMatch(des1, des2, k=2)
    good = [p[0] for p in knn if len(p) == 2 and p[0].distance < RATIO * p[1].distance]
    return good, len(knn)

def est_homography(kp1, kp2, good, thr):
    if len(good) < 4:
        return None
    src = np.float32([kp1[m.queryIdx].pt for m in good]); dst = np.float32([kp2[m.trainIdx].pt for m in good])
    H, mask = cv2.findHomography(src, dst, cv2.RANSAC, thr, maxIters=5000, confidence=0.995)
    if H is None:
        return None
    inl = mask.ravel().astype(bool)
    proj = cv2.perspectiveTransform(src[inl].reshape(-1, 1, 2), H).reshape(-1, 2)
    rmse = float(np.sqrt(np.mean(np.sum((proj - dst[inl]) ** 2, axis=1))))
    return H, inl, rmse

# C4. Warping and blending
MAX_CANVAS = 6000

def build_chain(Hpair, ref):
    """Hpair[i]: image i -> image i+1. Returns H_i->ref."""
    n = len(Hpair) + 1
    Hs = [None] * n; Hs[ref] = np.eye(3)
    for i in range(ref - 1, -1, -1):
        Hs[i] = Hs[i + 1] @ Hpair[i]
    for i in range(ref + 1, n):
        Hs[i] = Hs[i - 1] @ np.linalg.inv(Hpair[i - 1])
    return [H / H[2, 2] for H in Hs]

def warp_all(imgs, Hs):
```

```
    corners = []
    for im, H in zip(imgs, Hs):
        h, w = im.shape[:2]
        c = np.float32([[0, 0], [w, 0], [w, h], [0, h]]).reshape(-1, 1, 2)
        corners.append(cv2.perspectiveTransform(c, H).reshape(-1, 2))
    allc = np.vstack(corners)
    if not np.isfinite(allc).all():
        raise RuntimeError("Degenerate homography (infinite coordinates).")
    mn, mx = allc.min(0), allc.max(0)
    s = min(1.0, MAX_CANVAS / max(mx - mn))
    T = np.array([[s, 0, -mn[0] * s], [0, s, -mn[1] * s], [0, 0, 1]], dtype=np.float64)
    size = (int(math.ceil((mx[0] - mn[0]) * s)), int(math.ceil((mx[1] - mn[1]) * s)))
    warped, masks = [], []
    for im, H in zip(imgs, Hs):
        M = T @ H; h, w = im.shape[:2]
        warped.append(cv2.warpPerspective(im, M, size, flags=cv2.INTER_LINEAR))
        masks.append(cv2.warpPerspective(np.full((h, w), 255, np.uint8), M, size, flags=cv2.INTER_NEAREST))
    return warped, masks

def blend_feather(warped, masks):
    ws = [cv2.distanceTransform((m > 0).astype(np.uint8), cv2.DIST_L2, 3) for m in masks]
    num = sum(w[..., None] * im.astype(np.float32) for w, im in zip(ws, warped))
    den = sum(ws)[..., None]
    return np.clip(num / np.maximum(den, 1e-6), 0, 255).astype(np.uint8)

def _gauss_pyr(x, n):
    p = [x]
    for _ in range(n):
        p.append(cv2.pyrDown(p[-1]))
    return p

def _lap_pyr(x, n):
    g = _gauss_pyr(x, n)
    l = [g[i] - cv2.pyrUp(g[i + 1], dstsize=(g[i].shape[1], g[i].shape[0])) for i in range(n)]
    l.append(g[-1]); return l

def blend_multiband(warped, masks, levels=5):
    ws = np.stack([cv2.distanceTransform((m > 0).astype(np.uint8), cv2.DIST_L2, 3) for m in masks])
    win = np.argmax(ws, axis=0)
    covered = (np.stack(masks).max(0) > 0)
    hard = [((win == i) & covered).astype(np.float32) for i in range(len(warped))]
    lp = [_lap_pyr(im.astype(np.float32), levels) for im in warped]
    mp = [_gauss_pyr(h, levels) for h in hard]
    out_levels = []
    for l in range(levels + 1):
        num = sum(lp[i][l] * mp[i][l][..., None] for i in range(len(warped)))
```

```
        den = sum(mp[i][l] for i in range(len(warped)))[..., None]
        out_levels.append(num / np.maximum(den, 1e-6))
    res = out_levels[-1]
    for l in range(levels - 1, -1, -1):
        res = cv2.pyrUp(res, dstsize=(out_levels[l].shape[1],
            out_levels[l].shape[0])) + out_levels[l]
    res[~covered] = 0
    return np.clip(res, 0, 255).astype(np.uint8)

def run_full_pipeline(name, thr=STITCH_THR, blend="feather",
    ref=REF_IDX):
    """End-to-end: extract -> match -> RANSAC -> warp -> blend. Returns
        (panorama, info)."""
    t0 = time.perf_counter()
    feats, _ = extract(name, grays)
    Hpair, stats = [], []
    for i in range(N_IMG - 1):
        (k1, d1), (k2, d2) = feats[i], feats[i + 1]
        good, _ = match_pair(name, d1, d2)
        r = est_homography(k1, k2, good, thr)
        if r is None:
            raise RuntimeError(f"{name}: could not estimate H for pair
                {i+1}-{i+2} (matches={len(good)}).")
        Hpair.append(r[0]); stats.append(dict(good=len(good),
            inliers=int(r[1].sum()), rmse=r[2]))
    Hs = build_chain(Hpair, ref)
    warped, masks = warp_all(imgsC, Hs)
    pano = blend_feather(warped, masks) if blend == "feather" else
        blend_multiband(warped, masks)
    total_ms = (time.perf_counter() - t0) * 1000
    info = dict(Detector=name, Blend=blend,
                Keypoints_total=sum(len(f[0]) for f in feats),
                Initial_matches=sum(s["good"] for s in stats),
                    Inliers=sum(s["inliers"] for s in stats),
                Mean_Reproj_RMSE_px=float(np.mean([s["rmse"] for s in
                    stats])), Total_stitch_time_ms=total_ms)
    info["Inlier_ratio_pct"] = 100 * info["Inliers"] /
        max(info["Initial_matches"], 1)
    return pano, info

panos, rows_sum = {}, []
for name in DETECTORS:
    try:
        pano, info = run_full_pipeline(name, STITCH_THR, "feather")
        panos[name] = pano; rows_sum.append(info)
        cv2.imwrite(f"{OUT}/partC/panorama_{name}.jpg", pano)
    except Exception as e:
        print("FAILED:", name, "->", e)

# bonus: multi-band blending with the first detector
BONUS = DETECTORS[0]
pano_mb = None
try:
    pano_mb, info = run_full_pipeline(BONUS, STITCH_THR, "multiband")
    rows_sum.append(info)
    cv2.imwrite(f"{OUT}/partC/panorama_{BONUS}_multiband.jpg", pano_mb)
except Exception as e:
    print("Multi-band bonus failed:", e)
```

```
164
165 dfC_final = pd.DataFrame(rows_sum)[["Detector", "Blend",
        "Keypoints_total", "Initial_matches", "Inliers",
166                                     "Inlier_ratio_pct",
                                            "Mean_Reproj_RMSE_px",
                                            "Total_stitch_time_ms"]].round(3)
167 display(dfC_final);
        dfC_final.to_csv(f"{OUT}/tables/partC_detector_comparison.csv",
        index=False)
```